```
# STEP 1: IMPORT LIBRARY
import numpy as np


# STEP 2: DEFINE INPUT AND OUTPUT (AND GATE)
input_value = np.array([[0,0],
                        [0,1],
                        [1,1],
                        [1,0]])

output = np.array([0,0,1,0]).reshape(4,1)


# STEP 3: INITIALIZE WEIGHTS AND BIAS
weights = np.array([[0.1],
                    [0.3]])

bias = 0.2


# STEP 4: DEFINE SIGMOID FUNCTION
def sigmoid_func(x):
    return 1 / (1 + np.exp(-x))


# STEP 5: DEFINE DERIVATIVE FUNCTION
def der(x):
    return sigmoid_func(x) * (1 - sigmoid_func(x))


# STEP 6: TRAIN THE MODEL (FORWARD + BACKPROPAGATION)
for epochs in range(15000):

    # Forward pass
    weighted_sum = np.dot(input_value, weights) + bias
    first_output = sigmoid_func(weighted_sum)

    # Error calculation
    error = first_output - output
    total_error = np.square(first_output - output).mean()

    # Backpropagation
    first_der = error
    second_der = der(first_output)
    derivative = first_der * second_der

    # Gradient calculation
    t_input = input_value.T
    final_derivative = np.dot(t_input, derivative)

    # Update weights
    weights = weights - (0.05 * final_derivative)

    # Update bias
    bias = bias - (0.05 * np.sum(derivative))


# STEP 7: PRINT FINAL WEIGHTS AND BIAS
print("Weights:\n", weights)
print("Bias:\n", bias)


# STEP 8: TEST THE MODEL

# Test [1,0]
pred = np.array([1,0])
result = np.dot(pred, weights) + bias
print("[1,0] ->", sigmoid_func(result))

# Test [1,1]
pred = np.array([1,1])
result = np.dot(pred, weights) + bias
print("[1,1] ->", sigmoid_func(result))

# Test [0,0]
```

```
pred = np.array([0,0])
result = np.dot(pred, weights) + bias
print("[0,0] ->", sigmoid_func(result))

# Test [0,1]
pred = np.array([0,1])
result = np.dot(pred, weights) + bias
print("[0,1] ->", sigmoid_func(result))
```

```
Weights:
 [[6.62916366]
 [6.62916441]]
Bias:
 -10.231973157456265
[1,0] -> [0.02652435]
[1,1] -> [0.95375065]
[0,0] -> [3.59993686e-05]
[0,1] -> [0.02652437]
```